

נתונה משוואה: $\mathbf{Ax} = \mathbf{b}$ כאשר $\mathbf{A}$ מטריצה מסדר $n \times n$ ו $\mathbf{x}$, $\mathbf{b}$ וקטורי עמוד מסדר n .

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{pmatrix}$$

הצגה מפורשת של $\mathbf{A}$

נפתור את המשוואה בשני שלבים:

שלב א': אלימינציה של גאוס שהופכת את המטריצה למשולשית כלומר, איפוס האיברים שמתחת לאלכסון הראשי. איפוס האיברים נעשה על ידי מכפלה משמאל במטריצה $\mathbf{T}$ כך ש $\mathbf{TAx} = \mathbf{Tb}$. המטריצה $\mathbf{T}$ בנויה ממכפלה של מטריצות $\mathbf{T}_{ij}$ כשכל אחת מהן¹ מאפסת איבר אחד (האיבר ה- (i, j)):

$$\mathbf{T} = (\mathbf{T}_{n,n-1}) \cdots (\mathbf{T}_{n,3} \cdots \mathbf{T}_{5,3} \mathbf{T}_{4,3}) (\mathbf{T}_{n,2} \cdots \mathbf{T}_{4,2} \mathbf{T}_{3,2}) (\mathbf{T}_{n,1} \cdots \mathbf{T}_{3,1} \mathbf{T}_{2,1})$$

עמודה 1 עמודה 2 עמודה 3 עמודה $n-1$

כלומר סדר הפעולות הוא איפוס העמודה הראשונה אח"כ השניה וכו' עד העמודה ה- $n-1$. בכל עמודה מאפסים את האיברים אחד אחד החל מהאיבר הראשון שמתחת לאלכסון, עד האיבר שבשורה האחרונה. המטריצה $\mathbf{T}_{ij}$ שווה למטריצת היחידה בכל השורות למעט שורה i שם יש איבר נוסף בעמודה j אותו נסמן

$$\mathbf{T}_{i,j} = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & s_{i,j} & \cdots & 1 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & \cdots & \cdots & 1 \end{pmatrix}$$

כלומר $s_{i,j}$: שורה i ←

↑ עמודה j

¹הבהרה: $\mathbf{T}_{ij}$ היא מטריצה $n \times n$ ולא איבר (i, j) במטריצה $\mathbf{T}$

דוגמא למצב בו לא ניתן לבצע pivoting.
 מטריצה של 7×7 באמצע תהליך אלימינציה.
 איברים השונים מאפס מסומנים על ידי x.
 איבר זה מתאפס וכן גם כל העמודה
 שמתחתיו. רואים שיש 5 שורות עם 4
 איברים השונים מאפס – כלומר, שורות אלו
 תליות.

x	x	x	x	x	x	x
0	x	x	x	x	x	x
0	0	0	x	x	x	x
0	0	0	x	x	x	x
0	0	0	x	x	x	x
0	0	0	x	x	x	x
0	0	0	x	x	x	x

$$\mathbf{A}'\mathbf{x} = \mathbf{b}'$$

שלב שני: פתרון מערכת משולשית:

$$\mathbf{A}' = \begin{pmatrix} a'_{11} & a'_{12} & a'_{13} & \cdots & a'_{1n} \\ 0 & a'_{22} & a'_{23} & \cdots & a'_{2n} \\ 0 & 0 & a'_{33} & \cdots & a'_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a'_{nn} \end{pmatrix}$$

ובהצגה מפורשת

$$a'_{n,n} \cdot x_n = b'_n \Rightarrow x_n = \frac{b'_n}{a'_{n,n}}$$

את המשוואה האחרונה קל לפתור :

$$x_{n-1} = \frac{b'_{n-1} - x_n \cdot a'_{n-1,n}}{a'_{n-1,n-1}}$$

נציב במשוואה לפני האחרונה ונקבל:

$$x_i = \frac{b'_i - \sum_{k=i+1}^n a'_{i,k} \cdot x_k}{a'_{i,i}}$$

כלומר מקבלים: כאשר פותרים "אחורנית" מ: $i = n$ ל: $i = 1$ (שלב זה נקרא

הצבה חוזרת – back substitution).

מספר פעולות

מספר הפעולות הנדרשות לאלימינציה של גאוס:

פעולות חישוב ופעולות כפל לכל אברי המטריצה

$$\rightarrow 2 \cdot n \cdot (n-1) \quad a_{i,1} \text{ - איפוס עמודה ראשונה}$$

$$2 \cdot (n-1) \cdot (n-2) \quad a_{i,2} \text{ - איפוס עמודה שניה}$$

...

$$2 \cdot 2 \quad a_{i,n-1} \text{ - איפוס עמודה } n-1$$

$$\sum_{i=1}^{n-1} 2 \cdot (n-(i-1)) \cdot (n-i) = 2 \cdot \left(\sum_{i=1}^n (n-i)^2 + \sum_{i=1}^n (n-i) \right) = \frac{2}{3} \cdot n^3 + O(n^2) \quad \text{בסה"כ:}$$

בנוסף יש $O(n^2)$ פעולות לטיפול באיבר החופשי וכן $O(n^2)$ פעולות להצבה החוזרת.

כאשר רוצים לפתור את $\mathbf{Ax} = \mathbf{c}$ כאשר כבר פתרנו את $\mathbf{Ax} = \mathbf{b}$, אזי $\mathbf{A}$ כבר משולשית. אם שמרנו את

$s_{i,j}$ ניתן לבצע את הטרנספורמציה על $\mathbf{c}$ ואז לבצע את שלב ההצבה החוזרת כלומר, הפתרון נעשה ב- $O(n^2)$

פעולות.

פונקציה לאלימינציה של גאוס:

```
import numpy as np
def gauss_p(sa,sb):
    ni, nj = sa.shape
    x = np.zeros(ni)
    if ni != nj:
        print('not rectangular matrix')
        return x
    else:
        n = ni
        if n != sb.shape[0]:
            print('no vector b')
            return x
        a = np.zeros([n,n])
        a[:,:] = sa[:,:]
        b = np.zeros(n)
        b[:] = sb[::]

    eps = np.finfo(np.float).eps * np.amax(np.abs(np.diag(a)))
    for j in range (0, n-1):
        if abs(a[j,j]) < eps*1e5:
            if np.amax(np.abs(a[j+1:n,j])) < eps:
                print('zero pivot')
                return x
            loc = np.argmax(np.abs(a[j+1:n,j])) + 1
            if j == 0:
                a[j:j+1+loc:loc,j:] = a[j+loc::-loc,j:]
                b[j:j+1+loc:loc] = b[j+loc::-loc]
            else:
                a[j:j+1+loc:loc,j:] = a[j+loc:j-1:-loc,j:]
                b[j:j+1+loc:loc] = b[j+loc:j-1:-loc]
        for i in range (j+1, n):
            p = a[i,j] / a[j,j]
            a[i,j+1:n] -= p * a[j,j+1:n]
            b[i] -= p * b[j]
        for i in range (n-1, -1, -1):
            x[i] = (b[i] - np.sum(a[i,i+1:n]*x[i+1:n])) / a[i,i]
    return x
```

הערות:

- במהלך האלימינציה משנים את ערכי המטריצה $a(:, :)$ והווקטור $b(:)$, ולכן הם מועתקים בתחילת הפונקציה.
- ניתן לשנות את הפונקציה כך שיתאפשר להיכנס שוב עם מטריצה $a(:, :)$ שכבר עברה אלימינציה, ועם ווקטור $b(:)$ חדש. לכן לא משנים בפועל את האבר שמתאפס (יש לזכור לטפל ב-pivoting).
- במהלך החישוב מאבדים דיוק (לדוגמא כשעובדים עם $\text{real}(4)$ הדיוק מתקלקל עבור מטריצות של 50×50).
- ניתן לבצע pivoting בכל שורה (כלומר להחליף עם שורה בעלת איבר מקסימלי)

בז'קויות נא/ות

הפונקציה

מערב

הפונקציה

החלפת שורות

- pivoting

שני השורה שאפס

את אבר i, j - הפונקציה

לא מעצבים את אבר j

הצורה החדשה